

SciToolAgent 四个 Case 工具说明

目录

SciToolAgent 四个 Case 工具接入说明	2
1. 范围说明	2
2. 论文 Case I 需要的核心工具链	3
3. 与我们项目现有工具的映射	4
4. 推荐接入方式	5
5. 依赖和运行环境	8
6. ToolKG 扩展示例	9
7. 注册 adapter	10
8. 计划样例	11
9. 输出与论文结果对齐	13
10. 验证建议	14
11. Case I 风险与取舍	14
12. Case II: Chemical Reactivity Prediction And Analysis	15
13. Case III: Chemical Synthesis And Analysis	18
14. Case IV: MOF Materials Analysis	21
15. 四个 Case 的统一实施顺序	24
16. 统一注册与测试建议	25
17. 参考来源	27

SciToolAgent 四个 Case 工具接入说明

1. 范围说明

SciToolAgent 论文的 case studies 包括四个科学案例：

1. Protein design and analysis
2. Chemical reactivity prediction and analysis
3. Chemical synthesis and analysis
4. MOF materials analysis

其中只有第一个是蛋白质设计与分析。蛋白质案例内部又面向不同二级结构类别生成蛋白，例如 alpha、beta、alpha-beta。我们本地 `Cases.ipynb` 中实际演示的是 alpha protein 版本；论文图示和附录文本还涉及 alpha、beta、alpha-beta 多类别结果。

本文按“复现论文四个 scientific case studies”整理工具需求和接入方式，并在 Case I 中额外说明蛋白多类别复现路径。

本项目的工具调用链路是：

ToolKG -> Planner -> PlanStep.tool -> AdapterRegistry -> BaseToolAdapter -> StepResult

因此，四个 case 的任意工具要接入本项目，都需要完成：

1. 在 `src/kg/protein_tool_kg.json` 或新增 SciToolAgent KG 中声明工具节点。
2. 在 `src/adapters/` 中实现 `BaseToolAdapter`。
3. 在 `src/adapters/builtins.py` 中注册 adapter。
4. 在 `configs/tool_metadata/active_tool_metadata.json` 中补充成本、风险、可靠性等元数据。
5. 将工具输出整理为统一 `StepResult.outputs`，并给下游步骤提供稳定字段，例如 `result`、`csv_path`、`smiles`、`pdb_path`。

建议保留 SciToolAgent 原始工具名作为 `tool.id`，这样能直接对齐 notebook 工具链和论文叙述。

2. 论文 Case I 需要的核心工具链

根据论文 Case I 说明和本地 Cases.ipynb，蛋白设计与分析链路可抽象为：

```
Design protein sequence
-> Predict 3D structure
-> Analyze secondary structure
-> Predict unfolding force / energy
-> Calculate ANM vibrational modes
```

对应工具如下。

SciToolAgent 工具名	论文/Notebook 中的含义	推荐底层实现	我们项目当前状态
DesignProteinAlpha	生成 alpha 类蛋白序列	Chroma CATH conditional generation	需要新增，当前 protgpt2 只能近似替代，不能严格复现 CATH 条件设计
DesignProteinBeta	生成 beta 类蛋白序列	Chroma CATH conditional generation	需要新增
DesignProteinAlphaBeta	生成 alpha-beta 类蛋白序列	Chroma CATH conditional generation	需要新增；本地 notebook 检索到该工具
SequenceToPdb	将蛋白序列折叠为 PDB，并返回 pLDDT	ESMFold	可复用现有 esmfold / nim_esmfold / openfold adapter，但需要加 SciToolAgent 兼容别名
AnalyzeProteinStructure	分析 PDB 二级结构组成	DSSP + BioPython	可复用现有 dssp / biopython_qc，但需要输出 SciToolAgent 风格百分比

SciToolAgent 工具名	论文/Notebook 中的含义	推荐底层实现	我们项目当前状态
CalculateForceEnergyFromSequence	从序列预测 unfolding force / energy	ProteinForceGPT	需要新增 adapter 或远程服务
CalculateProteinANM	计算 ANM 前若干振动模态	ProDy ANM	需要新增 adapter，并添加 prody 依赖

本地 notebook 的 alpha 演示实际规划路径是：

```
DesignProteinAlpha
-> SequenceToPdb
-> AnalyzeProteinStructure
-> CalculateForceEnergyFromSequence
-> CalculateProteinANM
```

论文正文中描述的底层工具是 Chroma、ProteinForceGPT、ESMFold、ANM/ProDy、DSSP。

3. 与我们项目现有工具的映射

我们项目已有蛋白工具体系，核心文件包括：

- src/kg/protein_tool_kg.json
- src/adapters/builtins.py
- src/adapters/base_tool_adapter.py
- src/adapters/registry.py
- src/workflow/step_runner.py
- configs/tool_metadata/active_tool_metadata.json

现有可复用工具：

我们项目工具	可承担的 SciToolAgent	
	能力	说明
esmfold	SequenceToPdb	本地/Nextflow 结构预测路径
nim_esmfold	SequenceToPdb	远程 NVIDIA NIM ESMFold
openfold / OpenFold3 adapter	SequenceToPdb fallback	可作为结构预测备选
dssp	AnalyzeProteinStructure	二级结构注释
biopython_qc	部分结构/序列 QC	可辅助验证，但不是 DSSP 百分比输出的完整替代
protgpt2	DesignProtein* 弱替代	可生成序列，但不能保证 CATH alpha/beta/alpha-beta 条件

必须新增或补齐：

- ChromaCATHDesignAdapter
- ProteinForceGPTAdapter
- ProDyANMAdapter
- SciToolAgent 兼容别名 adapter，例如 SequenceToPdbAdapter 和 AnalyzeProteinStructureAdapter

4. 推荐接入方式

4.1 保留 SciToolAgent 原始工具名

建议在 ToolKG 中保留原始工具名：

DesignProteinAlpha

DesignProteinBeta

DesignProteinAlphaBeta

SequenceToPdb

AnalyzeProteinStructure
CalculateForceEnergyFromSequence
CalculateProteinANM

4.2 用兼容 wrapper 复用已有 adapter

对我们已经支持的能力，不需要重写底层逻辑，只需要加 wrapper：

SciToolAgent wrapper	内部委托
SequenceToPdb	ESMFoldAdapter、 NIMESMFoldAdapter 或 OpenFold3Adapter
AnalyzeProteinStructure	DSSPAdapter，必要时补充 BioPython 解析

例如 SequenceToPdb 的输出应整理为：

```
{  
  "pdb_path": "alpha_protein.pdb",  
  "plddt": 78.67,  
  "sequence": "...",  
  "result": "alpha_protein.pdb"  
}
```

AnalyzeProteinStructure 的输出应整理为 DSSP 二级结构百分比：

```
{  
  "secondary_structure_percent": {  
    "H": 85.5,  
    "B": 0.0,  
    "E": 0.0,  
    "G": 0.0,  
    "I": 0.0,  
    "T": 7.0,  
    "S": 1.5,  
  }  
}
```

```
    "P": 0.0,  
    "-": 6.0  
  },  
  "result": {  
    "H": 85.5,  
    "T": 7.0,  
    "-": 6.0  
  }  
}
```

4.3 对缺失工具新增 adapter

建议新增文件：

src/adapters/scitoolagent_protein_adapter.py

也可以拆成：

```
src/adapters/scitoolagent/  
├─ protein_design.py  
├─ protein_force.py  
├─ protein_structure.py  
└─ protein_anm.py
```

推荐工具到 adapter 的映射：

Adapter 类	tool_id	输入	输出
ChromaCATHDesignAdapter	DesignProteinAlpha , DesignProteinBeta, DesignProteinAlphaBeta	length、 cath_class	sequence、protein_filename、 cath_annotation
SequenceToPdbCompatAdapter	SequenceToPdb	sequence、 可选 protein_filename	pdb_path、 plddt
DSSPCompositionAdapter	AnalyzeProteinStructure	pdb_path	secondary_structure_percent

Adapter 类	tool_id	输入	输出
ProteinForceGPTAdapter	CalculateForceEnergyFromSequence	sequence	unfolding_force、energy
ProDyANMAdapter	CalculateProteinANM	pdb_path 或 protein_filename	eigenvalues、frequencies、normal_modes

5. 依赖和运行环境

5.1 Python 依赖

我们项目已经包含：

```
biopython
```

建议新增：

```
uv add prody
```

如果要本地运行 Chroma 和 ProteinForceGPT，还需要按对应项目安装模型依赖。由于这两个模型依赖可能较重，建议优先做远程服务化接入：

```
export CHROMA_PROTEIN_REST_BASE_URL=http://<host>:<port>
export PROTEINFORCEGPT_REST_BASE_URL=http://<host>:<port>
```

结构预测可继续沿用我们已有配置：

```
export NIM_API_KEY=...
export OPENFOLD3_REST_BASE_URL=http://<host>:<port>
```

5.2 外部二进制

AnalyzeProteinStructure 如果使用 DSSP，需要本机安装 DSSP 可执行文件。我们现有 DSSPAdapter 已能通过 healthcheck() 检查 binary 是否存在。

具体以服务器系统包管理器为准。

6. ToolKG 扩展示例

以 SequenceToPdb 为例, 可在 src/kg/protein_tool_kg.json 的 tools 中追加兼容节点:

```
{
  "id": "SequenceToPdb",
  "tool_id": "SequenceToPdb",
  "name": "SequenceToPdb",
  "domain": "protein/structure",
  "description": "SciToolAgent-compatible sequence-to-PDB wrapper backed by ESMFold or OpenFold",
  "capabilities": ["structure_prediction"],
  "io": {
    "io_type_id": "sequence_to_structure",
    "inputs": {
      "sequence": "str"
    },
    "outputs": {
      "pdb_path": "path",
      "plddt": "float",
      "result": "path"
    },
    "input_types": ["sequence"],
    "output_types": ["structure_pdb", "plddt"],
    "combinable": true
  },
  "constraints": {
    "preconditions": ["sequence_provided"],
    "resource_assumptions": ["esmfold_or_openfold_ready"],
    "limits": {
      "max_length": 2000
    }
  },
  "execution": "python",
  "cost_score": 0.65,
```

```

"safety_level": 1,
"priority": "P1",
"failure_modes": ["timeout", "remote_service_unavailable", "invalid_sequence"],
"preferred_next": ["AnalyzeProteinStructure", "CalculateProteinANM"],
"version": "1.0.0"
}

```

其他工具节点可按同样方式添加：

- DesignProteinAlpha: capabilities = ["sequence_generation"],
io_type_id = "goal_to_sequence_candidates"
- DesignProteinBeta: 同上, constraints.model_assumptions =
["cath_beta"]
- DesignProteinAlphaBeta: 同上, constraints.model_assumptions =
["cath_alpha_beta"]
- AnalyzeProteinStructure: capabilities = ["secondary_structure_annotation"]
- CalculateForceEnergyFromSequence: 新增 capability mechan-
ical_stability_prediction
- CalculateProteinANM: 新增 capability protein_dynamics_analysis

7. 注册 adapter

在 src/adapters/builtins.py 中注册：

```

from src.adapters.scitoolagent_protein_adapter import (
    CalculateProteinANMAdapter,
    ChromaCATHDesignAdapter,
    DSSPCompositionCompatAdapter,
    ProteinForceGPTAdapter,
    SequenceToPdbCompatAdapter,
)

```

```

def ensure_builtin_adapters() -> None:
    ...
    for tool_id, cath_class in (

```

```
        ("DesignProteinAlpha", "alpha"),
        ("DesignProteinBeta", "beta"),
        ("DesignProteinAlphaBeta", "alpha_beta"),
    ):
        try:
            get_adapter(tool_id)
        except KeyError:
            register_adapter(ChromaCATHDesignAdapter(tool_id, cath_class))

    for adapter in (
        SequenceToPdbCompatAdapter(),
        DSSPCompositionCompatAdapter(),
        ProteinForceGPTAdapter(),
        CalculateProteinANMAAdapter(),
    ):
        try:
            get_adapter(adapter.tool_id)
        except KeyError:
            register_adapter(adapter)
```

8. 计划样例

8.1 本地 notebook 中的 alpha protein 复现路径

```
{
  "task_id": "scitoolagent_case1_alpha",
  "steps": [
    {
      "id": "S1",
      "tool": "DesignProteinAlpha",
      "inputs": {
        "length": 200
      }
    },
  ],
}
```

```
    "id": "S2",
    "tool": "SequenceToPdb",
    "inputs": {
      "sequence": "S1.sequence",
      "protein_filename": "S1.protein_filename"
    }
  },
  {
    "id": "S3",
    "tool": "AnalyzeProteinStructure",
    "inputs": {
      "pdb_path": "S2.pdb_path"
    }
  },
  {
    "id": "S4",
    "tool": "CalculateForceEnergyFromSequence",
    "inputs": {
      "sequence": "S1.sequence"
    }
  },
  {
    "id": "S5",
    "tool": "CalculateProteinANM",
    "inputs": {
      "pdb_path": "S2.pdb_path",
      "num_modes": 10
    }
  }
]
}
```

8.2 论文 Figure 3 多类别复现路径

如果要复现论文中 alpha、beta、alpha-beta 三类结果，可按类别重复同一条链：

1. DesignProteinAlpha -> SequenceToPdb -> AnalyzeProteinStructure -> CalculateForceEnergyFromSequence -> CalculateProteinANM
2. DesignProteinBeta -> SequenceToPdb -> AnalyzeProteinStructure -> CalculateForceEnergyFromSequence -> CalculateProteinANM
3. DesignProteinAlphaBeta -> SequenceToPdb -> AnalyzeProteinStructure -> CalculateForceEnergyFromSequence -> CalculateProteinANM

工程实现上有两种方式：

1. 展开为 15 个 PlanStep，每个类别 5 步，最贴近 SciToolAgent 的 chain-of-tools。
2. 做 batch adapter，一次输入多个类别和长度，输出 `sequence_candidates`、`structure_results`、`anm_results`，更适合我们项目的实验矩阵。

9. 输出与论文结果对齐

最终 Summarizer 应至少汇总：

输出项	来源
protein sequence	DesignProtein*
PDB file path	SequenceToPdb
pLDDT	SequenceToPdb
secondary structure percentage	AnalyzeProteinStructure
unfolding force	CalculateForceEnergyFromSequence
unfolding energy	CalculateForceEnergyFromSequence
first 10 vibrational modes / eigenvalues	CalculateProteinANM

本地 alpha notebook 示例输出包括：

- sequence length: 200

- PDB 文件: `alpha_protein.pdb`
- pLDDT: 约 78.68
- 二级结构: H 约 85.5%, T 约 7.0%, 无序区约 6.0%
- unfolding force: 约 0.379
- energy: 约 0.371
- ANM modes: 前 10 个 eigenvalues

10. 验证建议

建议新增测试:

```
tests/unit/test_scitoolagent_protein_adapters.py
```

```
tests/integration/test_scitoolagent_case1_protein_chain.py
```

最小验证集:

1. DesignProteinAlpha 输入 `length=100`, 输出合法氨基酸序列。
2. SequenceToPdb 输入序列, 输出 `pdb_path` 和 `plddt`。
3. AnalyzeProteinStructure 输入 PDB, 输出 DSSP 百分比, 百分比总和接近 100。
4. CalculateForceEnergyFromSequence 输入序列, 输出 `force / energy` 数值。
5. CalculateProteinANM 输入 PDB, 输出 10 个 eigenvalues。
6. 端到端 alpha 链路能完成并进入 DONE。

执行命令:

```
uv run pytest tests/unit/test_scitoolagent_protein_adapters.py
```

```
uv run pytest tests/integration/test_scitoolagent_case1_protein_chain.py
```

```
uv run basedpyright src/adapters/scitoolagent_protein_adapter.py
```

11. Case I 风险与取舍

- Chroma 和 ProteinForceGPT 是复现论文 Case I 的关键; 如果只用 `protgpt2` 和简单规则替代, 只能算功能近似, 不能算严格复现。
- SequenceToPdb 可以优先复用我们已有结构预测 adapter, 这部分接入成本最低。

- `AnalyzeProteinStructure` 推荐走 DSSP, 而不是只用 BioPython PDB parser, 因为论文明确使用 DSSP/BioPython DSSP 做二级结构。
- `CalculateProteinANM` 需要 ProDy; 这是新增依赖, 但实现相对独立。
- 如果外部模型服务不可用, 应在 ToolKG readiness 中标记 degraded, 并允许 fallback 到现有 `protgpt2`、`esmfold`、`biopython_qc`, 但报告中要注明不是论文原始工具。

12. Case II: Chemical Reactivity Prediction And Analysis

Case II 的目标是: 读取 CSV 中的反应物 SMILES, 生成不同分子特征, 用机器学习模型预测反应活性, 并比较特征或算法表现。本地 notebook 中分成两轮实验。

第一轮用于比较特征:

```
GenerateRDKFingerprintsFromCSV
-> GenerateMorganfingerprintsFromCSV
-> GenerateElectricalDescriptorsFromCSV
-> MLPClassifier
-> MLPClassifier
-> MLPClassifier
```

第二轮用于比较算法:

```
GenerateElectricalDescriptorsFromCSV
-> MLPClassifier
-> AdaBoostClassifier
-> RandomForestClassifier
```

12.1 所需工具和推荐实现

SciToolAgent 工具			
具名	功能	推荐底层实现	接入状态
GenerateRDKFingerprintFromCSV	从 CSV 中的 SMILES 生成 RDKit/RDK fingerprint 特征	RDKit	需要新增
GenerateMorganfingerprintsFromCSV	生成 Morgan fingerprint 特征	RDKit Morgan-Generator	需要新增
GenerateElectricalDescriptorsFromCSV	生成电性/理化描述符	RDKit descriptors, 可扩展 Mordred	需要新增
MLPClassifier	训练/测试 MLP 分类器	scikit-learn	需要新增
AdaBoostClassifier	训练/测试 AdaBoost 分类器	scikit-learn	需要新增
RandomForestClassifier	训练/测试随机森林分类器	scikit-learn	需要新增
RXNMPredict	反应产物预测, retrieval 中出现但第一轮未执行	RXNMapper/RDKitCase III 也需要部模型	需要新增

12.2 推荐 adapter 设计

建议新增:

src/adapters/scitoolagent_chem_ml_adapter.py

推荐类:

Adapter 类	tool_id	输入	输出
CSVFingerprin tAdapter	GenerateRDKFingerprintsFrom CSV、GenerateMorganfingerpr intsFromCSV	csv_path、 smiles_c olumn	feature_csv_p ath、feature_ type、result
ElectricalDes criptorCSVAda pter	GenerateElectricalDescripto rsFromCSV	csv_path、 smiles_c olumn	feature_csv_p ath、descript or_names、 result
SklearnClassi fierAdapter	MLPClassifier、AdaBoostClas sifier、RandomForestClassif ier	feature_ csv_path、 label_co lumn	train_accurac y、test_accur acy、model_na me、result

输入 CSV 至少需要 SMILES 列和标签列，建议默认列名为 smiles 与 reactivity。特征工具输出 feature_csv_path，分类器工具读取该路径并输出 train_accuracy、test_accuracy。

12.3 依赖、ToolKG 与验证

建议新增依赖：

```
uv add rdkit scikit-learn pandas numpy
```

如需更丰富的 electrical descriptors，可选：

```
uv add mordredcommunity
```

新增 capability：

- molecular_feature_generation
- chemical_reactivity_classification

新增 io_type：

- csv_smiles_to_feature_csv
- feature_csv_to_classification_metrics

推荐测试:

```
tests/unit/test_scitoolagent_chem_features.py
tests/unit/test_scitoolagent_ml_classifiers.py
tests/integration/test_scitoolagent_case2_reactivity_chain.py
```

13. Case III: Chemical Synthesis And Analysis

Case III 的目标是: 给定两个反应物 SMILES, 预测反应产物, 将产物转 SELFIES, 生成文本描述, 并检查专利状态和爆炸性。

本地 notebook 的实际工具链:

```
RXNPredict
-> SMILEStoSELFIES
-> GenerateMoleculeDescription
-> CheckPatent
-> CheckExplosiveness
```

retrieval 阶段还召回了 TexToMoleculeSELFIES、RXNRetrosynthetic、Convert3DMolecules2SMILES、InChIKeyToSMILES、SELFIESToSMILES, 这些可作为第二阶段扩展。

13.1 所需工具和推荐实现

SciToolAgent 工			
具名	功能	推荐底层实现	接入状态
RXNPredict	反应产物预测	IBM RXN、 RXNMapper + 模板、或远程服 务	需要新增
SMILEStoSELFIES	SMILES 转 SELFIES	selfies Python 包	需要新增
GenerateMoleculeDescription	根据 SELF- IES/SMILES 生 成文字描述	LLM adapter 或 规则模板	需要新增

SciToolAgent 工具			
具名	功能	推荐底层实现	接入状态
CheckPatent	查询分子是否已被专利覆盖	外部专利 API / 本地缓存 / LLM 搜索摘要	需要新增
CheckExplosiveness	根据 CAS/SMILES 判断爆炸性	本地毒性/危险品数据 + PubChem/CAS 查询	需要新增
SMILESToCAS	SMILES 到 CAS	PubChem / ChemSpider / 本地缓存	Case IV 也需要

13.2 推荐 adapter 设计

建议新增：

src/adapters/scitoolagent_synthesis_adapter.py

推荐类：

Adapter			
类	tool_id	输入	输出
ReactionPredictionAdapter	RXNPredict	reactant_smiles 或 reaction_smiles	product_smiles、reaction_summary、result
SelfiesConversionAdapter	SMILEStoSELFIES、SELFIESToSMILES	smiles, selfies	selfies, smiles、result
MoleculeDescriptionAdapter	GenerateMoleculeDescription	smiles、selfies	description、result
PatentLookupAdapter	CheckPatent	smiles、cas、name	patent_status、evidence、result

Adapter			
类	tool_id	输入	输出
Explosive	CheckExplosive	smiles、cas	explosive、cas、evidence、result
nessLooku	ness		
pAdapter			

RXNPredict 输出建议统一为:

```
{
  "product_smiles": "O=C1CCC(=O)O1",
  "result": "O=C1CCC(=O)O1"
}
```

13.3 依赖、ToolKG 与验证

建议新增依赖:

```
uv add rdkit selfies pubchempy
```

如果使用外部反应预测或专利 API:

```
export RXN_PREDICT_BASE_URL=http://<host>:<port>
export PATENT_LOOKUP_API_KEY=...
export CHEMSPIDER_API_KEY=...
```

新增 capability:

- reaction_prediction
- molecule_representation_conversion
- molecule_description_generation
- chemical_patent_lookup
- chemical_hazard_lookup

推荐测试:

```
tests/unit/test_scitoolagent_synthesis_adapters.py
```

```
tests/integration/test_scitoolagent_case3_synthesis_chain.py
```

如果 RXNPredict 暂时无法接入真实模型，可以先固定一条示例 reaction 的本地 fixture，但报告中必须注明这是 fixture fallback。

14. Case IV: MOF Materials Analysis

Case IV 的目标是：给定 MOF 的 CIF 文件，预测热稳定性和 CO₂ 吸附，将 MOF 转成 SMILES，再转 CAS 并查询价格。

本地 notebook 的实际工具链：

```
PredictStability
-> PredictAdsorption
-> MOFToSMILES
-> SMILESToCAS
-> CASToPrice
```

retrieval 阶段还召回了 GetFloatingSolventMolecules、MofLattice、MofFractionalCoordinates、GetTerminalIndices、MofNeighborIndices，这些属于 MOF 结构解析辅助工具，可放在第二阶段接入。

14.1 所需工具和推荐实现

SciToolAgent 工			
具名	功能	推荐底层实现	接入状态
PredictStability	预测 MOF 热稳定性/溶剂移除稳定性	MOFTransformer / 原 SciToolAgent ANN 模型 / 远程服务	需要新增
PredictAdsorption	预测 CO ₂ /H ₂ 吸附	预训练吸附模型 / 远程服务	需要新增
MOFToSMILES	从 CIF 提取 linker 或近似 SMILES	pymatgen + mofid/mofdscribe 或原工具服务	需要新增

SciToolAgent 工具			
具名	功能	推荐底层实现	接入状态
SMILESToCAS	SMILES 到 CAS	PubChem/ChemSpider/ 本地缓存	可作为 Case III 的 identifier adapter
CASToPrice	CAS 到平均价格	本地缓存 / 供应 商 API / web API	需要新增
MofLattice	读取晶格参数	pymatgen	可作为辅助工具 新增
MofFractionalCoordinates	读取分数坐标	pymatgen	可作为辅助工具 新增

14.2 推荐 adapter 设计

建议新增：

src/adapters/scitoolagent_mof_adapter.py

推荐类：

Adapter 类	tool_id	输入	输出
MOFStabilityAdapter	PredictStability	cif_path	thermal_stability、 sol- vent_removal_stability、 result
MOFAdsorptionAdapter	PredictAdsorption	cif_path、 可选 gas 条件	co2_absolute_mg_g、 h2_absolute_mg_g、 result
MOFToSmilesAdapter	MOFToSMILES	cif_path	smiles、status、 result
SmilesToCASAdapter	SMILESToCAS	smiles	cas、result
CASToPriceAdapter	CASToPrice	cas	average_price、 currency、result

Adapter 类	tool_id	输入	输出
MOFStructureParserAdapter	MofLattice、 MofFractional- Coordinates 等	cif_path	晶格/坐标/邻接信息

PredictStability 输出建议:

```
{
  "filename": "HKUST-1.cif",
  "thermal_stability": 324.049896,
  "solvent_removal_stability": 0.948832,
  "result": {
    "thermal_stability": 324.049896,
    "solvent_removal_stability": 0.948832
  }
}
```

PredictAdsorption 输出建议:

```
{
  "filename": "HKUST-1.cif",
  "finished": true,
  "co2_absolute_mg_g": 106.885632,
  "h2_absolute_mg_g": 0.052386,
  "result": {
    "co2_absolute_mg_g": 106.885632
  }
}
```

14.3 依赖、ToolKG 与验证

建议新增:

```
uv add pymatgen pandas numpy
```

可选:

```
uv add mofdscribe
```

MOF 稳定性和吸附预测如果没有轻量本地模型，建议服务化：

```
export MOF_STABILITY_REST_BASE_URL=http://<host>:<port>
export MOF_ADSORPTION_REST_BASE_URL=http://<host>:<port>
export CHEM_PRICE_API_KEY=...
```

新增 capability：

- mof_stability_prediction
- mof_adsorption_prediction
- mof_structure_conversion
- chemical_price_lookup

推荐测试：

```
tests/unit/test_scitoolagent_mof_adapters.py
tests/integration/test_scitoolagent_case4_mof_chain.py
```

如暂时没有真实 MOF 预测模型，建议将 PredictStability 和 PredictAdsorption 标记为 degraded，使用 fixture 输出跑通链路，但不要把 fixture 结果写成真实实验结论。

15. 四个 Case 的统一实施顺序

建议按依赖难度分阶段接入：

阶段	优先工具	覆盖 Case	理由
P0	SequenceToPdb、 AnalyzeProtein- Structure、 SMILEStoSELFIES、 SMILEStoCAS、 RDKit 特征工具	Case I/II/III/IV	可复用现有结构 预测/DSSP， RD- Kit/SELFIES 接 入成本低

阶段	优先工具	覆盖 Case	理由
P1	MLPClassifier、 AdaBoostClassifier、 RandomForestClassifier、 CalculateProteinANM	Case I/II	scikit-learn 和 ProDy 相对独立，容易验证
P2	CheckExplosiveness CASToPrice、 GenerateMoleculeDescription、 MOFToSMILES	Case III/IV	需要本地数据或外部查询，但可做缓存
P3	DesignProtein*、 CalculateForceEnergyFromSequence、 RXNPredict、 PredictStability、 PredictAdsorption	Case I/III/IV	依赖较重模型或远程服务，是严格复现的关键

16. 统一注册与测试建议

建议新增一个 SciToolAgent 工具清单：

src/adapters/scitoolagent_tools.py

内容按 case 分组：

```
SCIToolAgent_CASE1_TOOLS = (
    "DesignProteinAlpha",
    "DesignProteinBeta",
```

```
        "DesignProteinAlphaBeta",
        "SequenceToPdb",
        "AnalyzeProteinStructure",
        "CalculateForceEnergyFromSequence",
        "CalculateProteinANM",
    )

    SCITOOLAGENT_CASE2_TOOLS = (
        "GenerateRDKFingerprintsFromCSV",
        "GenerateMorganfingerprintsFromCSV",
        "GenerateElectricalDescriptorsFromCSV",
        "MLPClassifier",
        "AdaBoostClassifier",
        "RandomForestClassifier",
    )

    SCITOOLAGENT_CASE3_TOOLS = (
        "RXNPredict",
        "SMILEStoSELFIES",
        "GenerateMoleculeDescription",
        "CheckPatent",
        "CheckExplosiveness",
    )

    SCITOOLAGENT_CASE4_TOOLS = (
        "PredictStability",
        "PredictAdsorption",
        "MOFTtoSMILES",
        "SMILEStoCAS",
        "CASToPrice",
    )
```

再在 `ensure_builtin_adapters()` 中按 adapter 类型批量注册。

建议新增：

```
tests/unit/test_scitoolagent_case_adapters.py
tests/integration/test_scitoolagent_case1_protein_chain.py
tests/integration/test_scitoolagent_case2_reactivity_chain.py
tests/integration/test_scitoolagent_case3_synthesis_chain.py
tests/integration/test_scitoolagent_case4_mof_chain.py
```

每个 integration test 都应验证：

1. 所有 PlanStep.tool 均存在于 ToolKG。
2. ensure_built_in_adapters() 后所有工具均可 get_adapter(tool_id)。
3. 每一步输出字段可被下一步引用，例如 S1.feature_csv_path、S2.product_smiles、S3.smiles。
4. 失败时能返回统一错误分类，不直接绕过 workflow 状态机。

17. 参考来源

- [SciToolAgent paper summary and case-study description](#)
- 本项目工具接入基类：src/adapters/base_tool_adapter.py
- 本项目 adapter 注册表：src/adapters/registry.py
- 本项目内置 adapter 注册：src/adapters/builtins.py
- 本项目 ToolKG：src/kg/protein_tool_kg.json